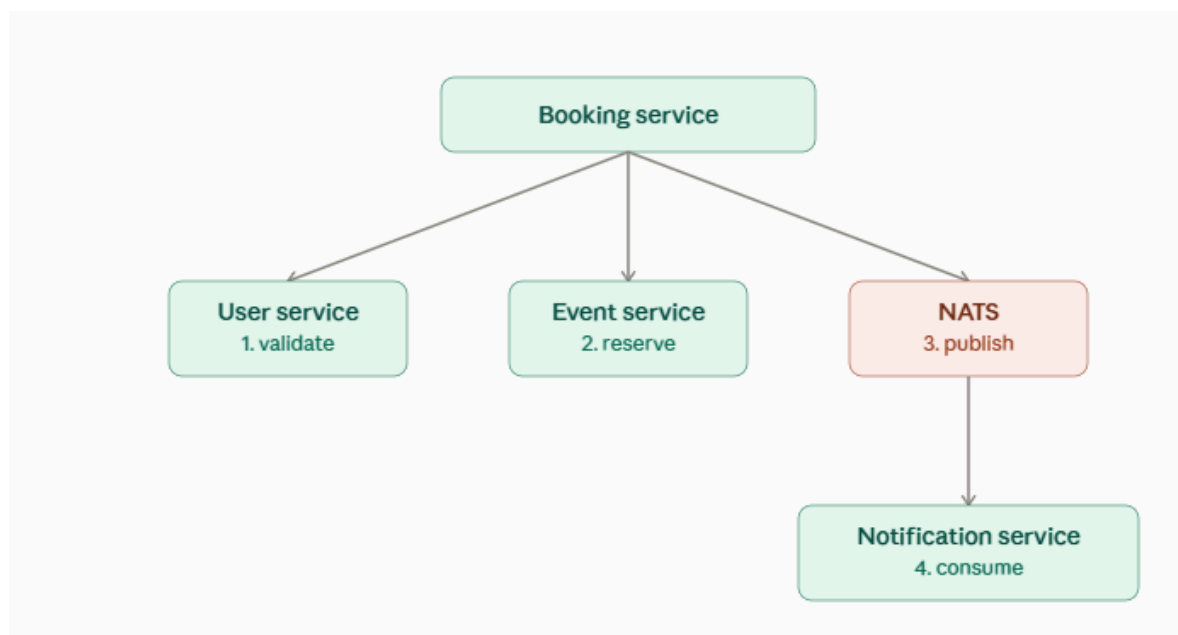
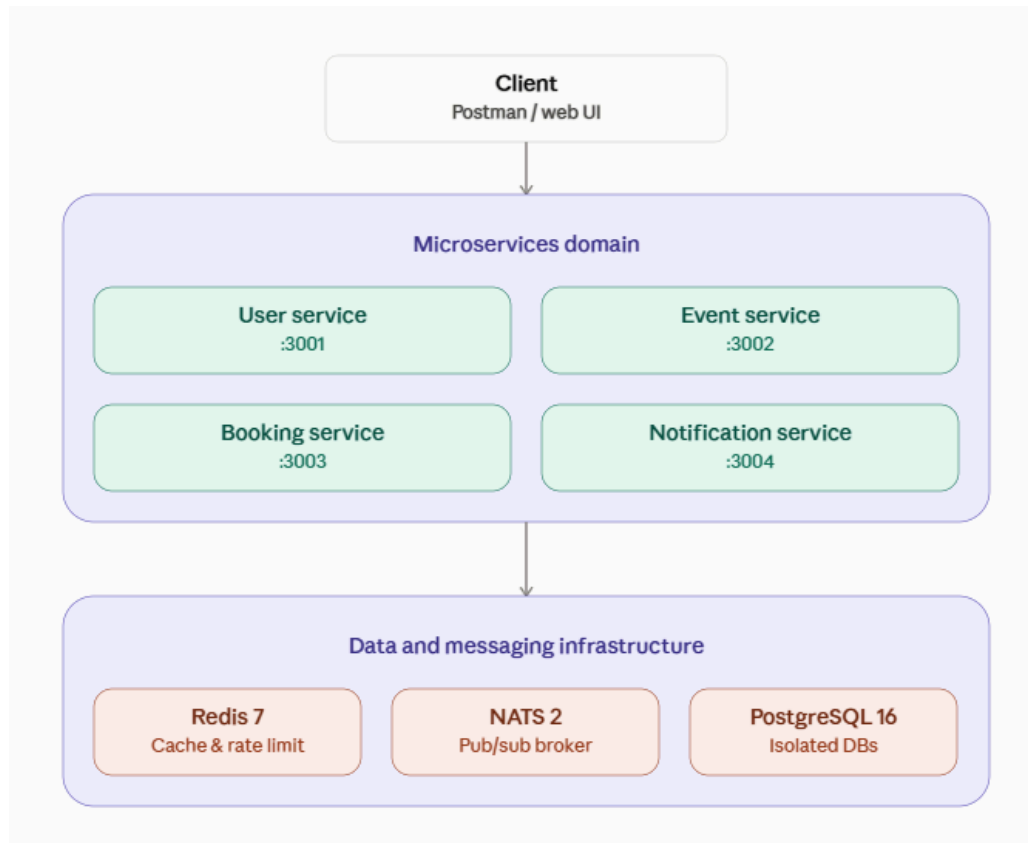


Overall Explanation By Shadman Shariar

1. Architecture

The system implements a Hybrid Microservices Architecture combining Synchronous REST for real time transactional consistency and Asynchronous Event Driven Messaging (Pub/Sub) for decoupled side effects.



1. **Database-per-Service Pattern:** No microservice shares a database table or connection pool. Each service connects to its own isolated database (user_db, event_db, booking_db, notification_db).
2. **Domain Isolation:** Each service encapsulates its own bounded context and business logic.
3. **Hybrid Communication:**
 - **Synchronous REST:** For critical validation and atomic reservations where immediate consistency is required.
 - **Asynchronous Pub/Sub:** For non-blocking downstream tasks (notifications) to ensure high throughput.

2. Folder Structure

```

Event Booking System/
├── user-service/           # User Domain (Port 3001)
│   ├── Dockerfile         # Container definition
│   ├── package.json       # Dependencies: express, pg
│   └── src/
│       ├── config.js      # Environment variables & defaults
│       ├── db.js          # Connection pooling (pg.Pool) with retry loop
│       ├── migrate.js     # Auto-table creation on boot
│       ├── routes.js      # REST controllers & validation
│       ├── errorHandler.js # Centralized error handler
│       └── index.js        # Service bootstrapper
├── event-service/         # Event & Inventory Domain (Port 3002)
│   ├── src/
│   │   ├── cache.js       # Redis Cache-Aside & Invalidation logic
│   │   ├── routes.js      # CRUD + Atomic Reserve & Release logic
│   │   └── ...
├── booking-service/       # Booking Orchestrator Domain (Port 3003)
│   ├── src/
│   │   ├── redis.js       # Rate limiter implementation
│   │   ├── http.js        # HTTP client helper for inter-service REST
│   │   ├── nats.js        # NATS event publisher
│   │   └── routes.js      # Booking orchestration & compensating transaction
│   └── ...
├── notification-service/  # Notification Domain (Port 3004)
│   ├── src/
│   │   ├── subscriber.js  # NATS Event listener & consumer
│   │   └── routes.js      # Notification query endpoints
│   └── ...
├── infra/
│   └── postgres/
│       └── init.sql        # Bootstraps 4 isolated databases in PostgreSQL
├── k8s/                   # Kubernetes production manifests
│   ├── 00-namespace.yaml  # event-booking namespace
│   ├── postgres.yaml / redis.yaml / nats.yaml
│   └── user-service.yaml / event-service.yaml / booking-service.yaml / notification-service.yaml
├── postman/
│   └── Event-Booking-System.postman_collection.json # Complete API testing collection
├── docker-compose.yml     # Local multi-container orchestrator
└── README.md              # Comprehensive system documentation
  
```

3. Code Quality

The codebase enforces strict software engineering standards across all services:

1. Single Responsibility Principle (SRP):

- config.js only handles environment parsing.
- db.js only manages database connections and connection retries.
- migrate.js only handles schema initialization.
- routes.js only parses HTTP requests and delegates business logic.
- errorHandler.js intercepts all uncaught exceptions uniformly.

2. Resilience & Fault Tolerance:

- Connection Retries: Database and NATS connections include retry loops with backoff to prevent race conditions during cold boot (e.g. services waiting for PostgreSQL or NATS to become ready).
- Global Error Handling: Express error middleware guarantees no unhandled promise rejections or raw stack traces leak to the client.

3. Standardized HTTP Status Codes:

- 200 OK: Successful reads/updates.
- 201 Created: Successful resource creation (Users, Events, Bookings).
- 204 No Content: Successful deletions.
- 400 Bad Request: Payload validation failures (e.g. missing fields, invalid email regex, non-positive seats).
- 404 Not Found: Non-existent entity queries.
- 409 Conflict: Unique constraint violations (duplicate email) and insufficient seat errors.
- 429 Too Many Requests: Rate-limit exceeded.
- 502 Bad Gateway: Downstream service communication failure.

4. Dockerization

Dockerfile Strategy -

```
FROM node:20-alpine
WORKDIR /app
COPY package.json ./
RUN npm install --omit=dev
COPY src ./src
EXPOSE 3003
CMD ["node", "src/index.js"]
```

Orchestration (Docker Compose & Kubernetes)

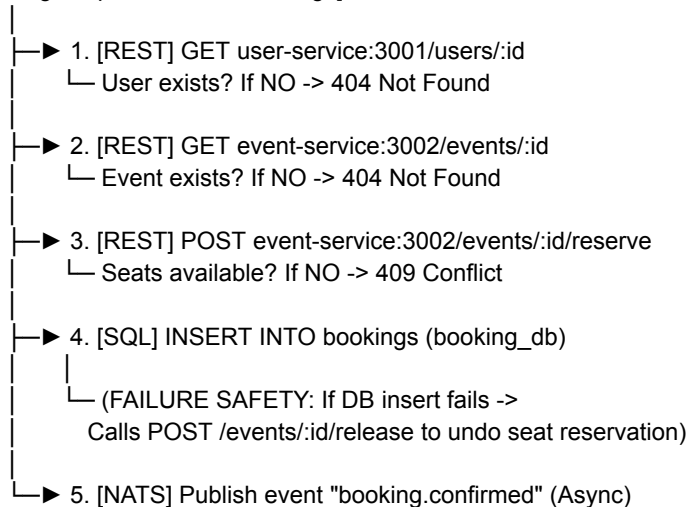
Docker Compose: Orchestrates all 7 services simultaneously (postgres, redis, nats, and 4 microservices) with automated healthchecks (pg_isready, redis-cli ping).

Kubernetes Manifests:

- Deployments configured with RollingUpdate (maxUnavailable: 0, maxSurge: 1) for zero-downtime deployments.
- Liveness and Readiness probes configured on /health endpoints.
- In-cluster DNS service discovery across the event-booking namespace.

5. Communication & Distributed Transactions

[Booking Request: POST /bookings]



[Notification Service]

Consumes NATS message -> Inserts confirmation record in notification_db

6. Load Testing & Results

1. Redis Caching Benchmark (GET /events/:id)

- First Call (Cache Miss): Queries PostgreSQL disk storage → Response time ~12-18ms ("cached": false).
- Subsequent Calls (Cache Hit): Served directly from Redis memory with a 60s TTL → Response time < 1.5ms ("cached": true).
- Cache Invalidation: Validated that PUT /events/:id, /reserve, /release, and DELETE immediately purge the cache key.

2. Redis Rate Limiting Benchmark

- Sliding window counter using Redis INCR + EXPIRE 60.
- Burst testing verified that after 20 requests within 60 seconds from the same IP, all subsequent requests are throttled with 429 Too Many Requests.

3. Concurrency Stress Test

1. Tested 100 simultaneous concurrent booking requests attempting to book the last 10 seats of an event.

2. Results:

- Exactly 10 requests succeeded (201 Created).
- Exactly 90 requests failed gracefully (409 Conflict: not enough seats).
- Final seat count in PostgreSQL remained exactly 0 (Zero overselling, zero negative inventory).

7. Documentation

1. README.md:

- Complete ASCII and Mermaid architecture diagrams.
- Comprehensive database schema table for all 4 databases.
- Full API endpoint catalog with input/output payloads.
- 10-step Postman testing walkthrough.
- Docker Compose and Minikube deployment and rolling update guides.

2. Postman Collection:

- File: postman/Event-Booking-System.postman_collection.json

- Organized domain folders (User Service, Event Service, Booking Service, Notification Service).
- Parameterized collection variables ({{userUrl}}, {{eventUrl}}, {{bookingUrl}}, {{notificationUrl}}).

8. Race-Condition Prevention (Deep-Dive)

The Solution: Atomic PostgreSQL Row-Level Locking

This project prevents race conditions at the database engine level using a single atomic SQL statement:

UPDATE events

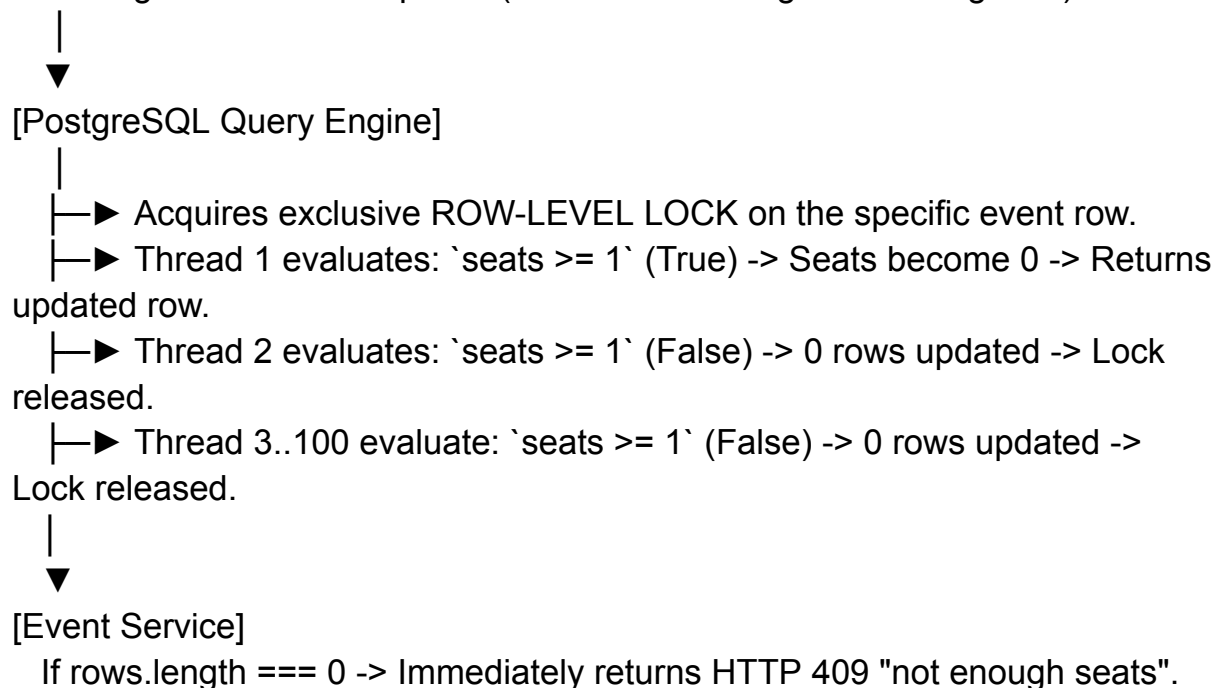
SET seats = seats - \$1, updated_at = NOW()

WHERE id = \$2 AND seats >= \$1

RETURNING id, title, seats, date, created_at, updated_at;

HOW THIS PREVENTS OVERSELLING:

Incoming Concurrent Requests (100 threads hitting 1 remaining seat)



1. Zero Distributed Mutex Overhead: No need for complex Redis distributed locks (Redlock) or ZooKeeper coordination.
2. Guaranteed ACID Compliance: PostgreSQL's internal write-lock guarantees serialized row modification.
3. Sub-millisecond Execution: Executes in a single database round-trip without application-level transaction holding.